CPSC 3660 Project Report

Winter 2026

Members:
Andriy Zynyuk - 001242739
Ronan Kunkel - 001245952
James - 001240091
Kyle Bjerstedt - 001228726

How to Use:

1. Log in as Admin, Employee, Customer.
2. Add information to anything available to you. Ex. you are a Customer trying to see the payment history and vehicle information as well as make a payment on a vehicle. sign in with your ID, select your vehicle in the top right of the page. Once the vehicle is selected you can click on the labels at the top of the page to be brought to the page relevent to seeing the information on the vehicle as well as the payment history. Finally, you can click the Make Payment button to be brought to the page that allow you to fill in the amount you are paying and Bank ID the payment will come from. Hit the Submit Payment button at the bottom and the database will save your payment.
3. Log out.

Development Stages:

1. Overall Design and ER Diagram design Our first step was getting together as a group, trying to understand the project requirements, and what we needed to do. We spent quite a bit of time dissecting the different input forms that the project used to see how they would look like in a DataBase. At this stage, we created a simple ER diagram to track our progress, and to help us later when creating the DB schema. We listed all of the different tables we will need for the project, as well as the different attributes that each table will have. At this point we also tried to figure out what primary and foreign keys each table will rely on.

2. Creating SQL Schema and Inserting Data into DB Now that we had a decent undestanding of what tables and attributes we will have in our DataBase, we created a DB using XAMPP, and added the tables into it. At the same time, we created some sample data to populate the tables, and to have data to test and work with.

3. Testing PHP and SQL together With having a DataBase to work with, we could now start testing it on a real website. We created a simple php file that would connect to our XAMPP DB, and would allow us to interact with it. Our first website simply displayed all the vehicles in our vehicle table. This showed us that the connection was successful, and that we could continue developing our project further. At this stage we also tested out some SQL queries such as inserting a new car into the db to make sure that everything worked.

4. Implementing Forms from Jones Auto At this stage we started implementing the sample forms provided in the project description. This included the car purchase, car sale, warranty, and payment forms. We made sure that each form contained all of the same information that the forms in the project description had. We also made sure that each form would properly send the data into our DB.

5. Creating Views We started creating some output views that the user will be able to interact with, such as monthly report and inventory. These were simple views that would retrieve some data from the DB, and display them to the user. For example, the inventory view would grab all vehicles that are not in the sale table, and display them to the user.

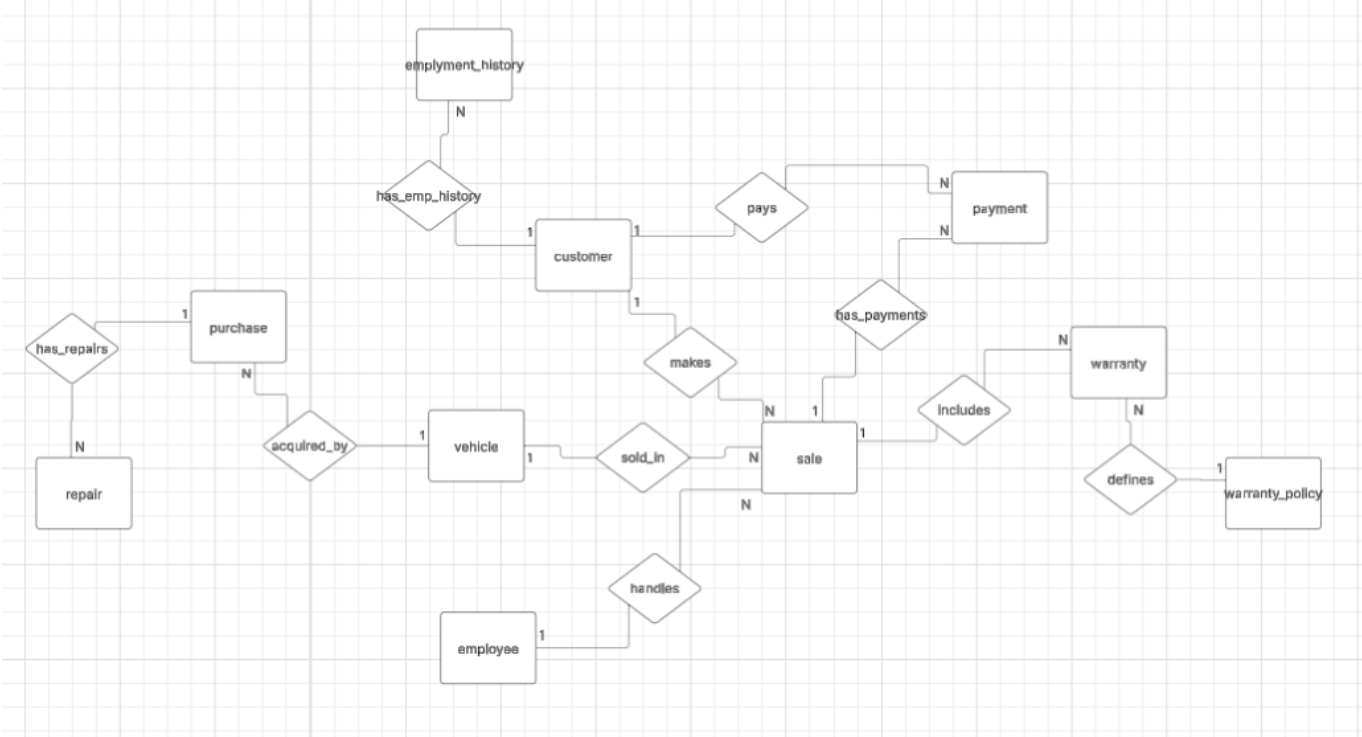
6. Refactoring Code Now that our project started to come together, we decided to refactor the code early, so we wouldn't have to do it later. Code refactor included sorting the various project files into seperate folders, as well as making sure that the code was consistent, readable, and clean.

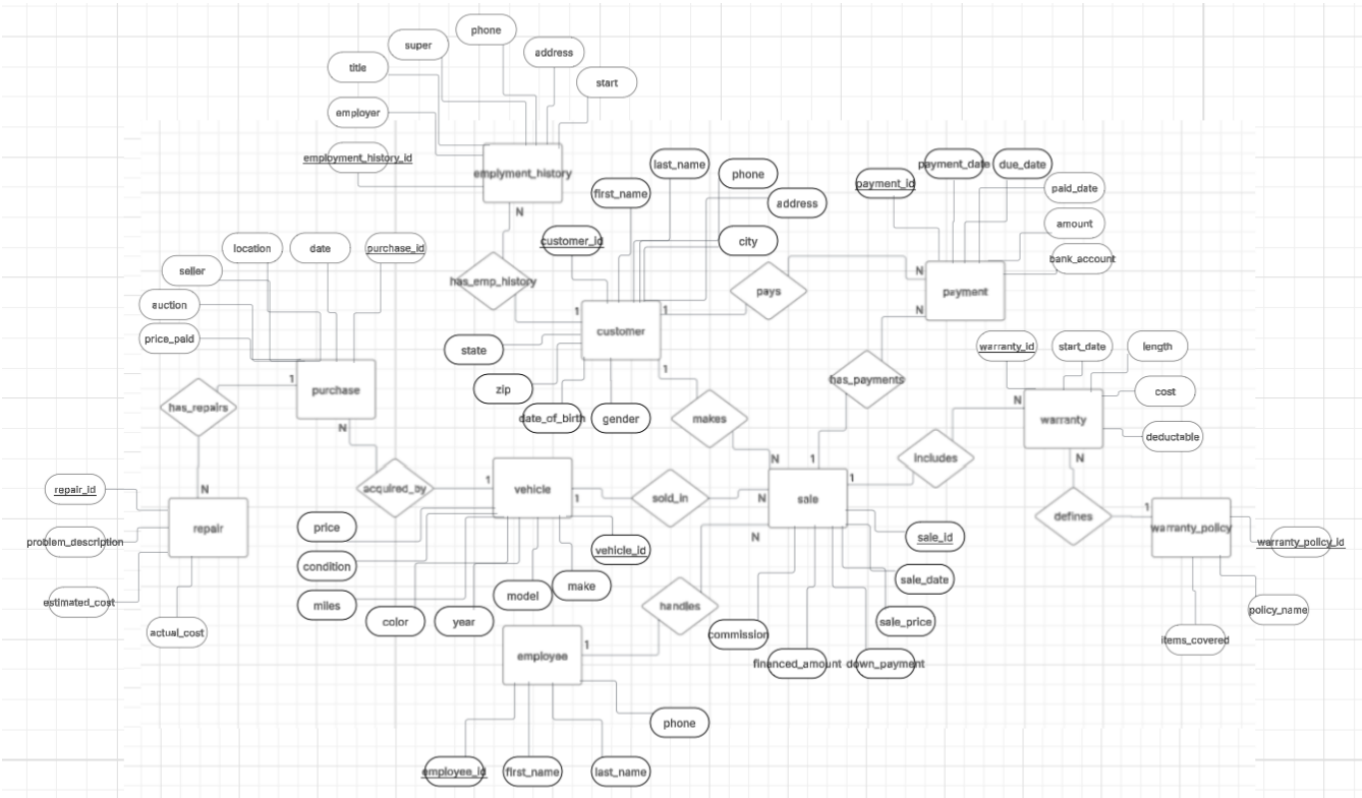
7. Adding a Login System We wanted to seperate the project into three different user types: customer, employee, and admin. This was done so that we wouldn't have a massive dashboard of various functionality that wasn't closely connected, and to make the project more realistic. The customer is able to view their vehicle(s), as well as make a payment. The employee is able to purchase and sell cars, as well as adding warranty, and adding customers, and have some views of the DB. The admin has pretty much all of the functionality as the customer and employee, but has more views, and is able to add a new employee.

8. Creating More Views We developed more views that would be useful for the users of this project such as payments, customers, and vehicles and inventory. All of these views are also able to be filtered by various ways. For example, the vehicles can be filtered by their make, model, years, miles and book price. This would allow the employee to find the perfect vehicle for a customer for example.

9. Testing and Polishing Project After getting the functionality of the website down, we extensively tested everything to make sure it works, and interacts correctly with our DB. At this point we were fixing bugs and polishing the project down to get it ready to hand in.

ER Diagram:





Input Forms:

1. Purchase Car
2. Sell Car
3. Add Warranty to Car
4. Add Payment to Car
5. Add Customer
6. Add Employee

Output Forms:

1. Inventory
2. All Vehicles
3. Vehicle Info
4. Payment History
5. Customers
6. Employees
7. Monthly Report

Assumptions Made:

1. The dealership will not buy the same car twice.

What we Learned:

1. Creating and Using DB
2. Using XAMPP
3. Connecting PHP and SQL

Project Improvements:

1. Cleaner Code Although we tried to keep our code clean and organized, we always improve. For example, in the schema, some attributes are camel case, while others are snake case.

2. Normalization As we learned normalization later in the semester, we were not able to apply these skills earlier on when we started on the project, which led to a DB which could use some normalization. For example, the vehicle purchase table contains auction, location, and seller. If the company buys from a single auction often, then there would be quite a bit of redundant data in the tables. This table could be split into purchase and seller for example instead.